

LL(1)->or every first pos of non-terminals, write their transition number. if first pos is epsilon, then write in its follow pos $S \rightarrow \epsilon$.

LR0->Reduce in every ending.

SLR2-> same as LR0, but Reduced only in every follow pos of terminal for each reduced i.

CLR1->Reduced will be in lookahead symbol only, lookahead = first pos of symbol after explorable symbol.

LALR1-> Merge those states where, Everything is same, but lookahead symbol is different and generate a new parsing table.

Operator grammar = No epsilon in the grammar and no adjacent Non-Terminals.

some Non-operator grammar can be changed to operator grammar.

A dead or Trap State is a non-final state in a DFA from which no sequence of inputs will ever lead to an accepting state. Means, once the automation enters a dead state, it loops back to itself for all possible input symbols.

A grammar is ambiguous if it has at least one such string in its language which has either multiple leftmost derivation or multiple rightmost derivation or multiple parse tree. Example,

$E \rightarrow E + E \mid E * E \mid id$. The string $id + id * id$ makes the grammar ambiguous.

Ardens Lemma is a mathematical theorem in formal language theory used to solve linear regular expression equations and convert finite automata into equivalent regular expressions.

Parse tree is a graphical representation of a derivation that shows how a start symbol of a grammar generates a specific string in the language.

It contains:

Root node: Represents the start symbol of the grammar.

Intermediate node: Represents the Non-Terminal symbols expanded during derivation.

Leaf nodes: Represents the Terminal symbols or epsilon which yields the derived string in the language if read left to right.

BNF(Backus-Naur Form) is a formal notation used to describe the syntax of programming languages, context free grammars and data formats.

Key Elements:

Non-terminals:

Terminals:

Production Rule Symbol:

Or Symbol:

Pushdown automaton is a computational model used to recognize context free languages. It is essentially a Finite Automaton augmented with an auxiliary memory mechanism structured as a Stack (a Last-In, First-Out memory structure).

$a+(b*c+b*c+d)$ to TAC

$T1 = b*c$

$T2 = b*c$

$T3 = T1 + T2$

$$T4 = T3 + d$$

$$T5 = a + T4$$

Code optimization is the process of improving the intermediate code so that the target code runs faster, uses less memory, or consumes less power.

Key Techniques:

Dead Code Elimination: Removing code that has no effect.

Common Subexpression Elimination: Avoiding redundant computations.

Loop Optimization: Moving invariant computations outside loops.

A symbol table is a data structure used in compiler design to store the information about identifiers(Variable, Functions, Classes etc) used in a program.

It acts like a database for the compiler.

it holds:

Identifier names, Data Type, Scope and Memory address.

Type conversion is the process of translating a value from one data type to another to ensure in an expression variables are evaluated consistently.

Token is an abstract symbol representing a lexical unit. It represents a meaningful sequence of characters in the source program. e.g. Keywords, identifiers, operators.

A pattern is a rule that describes the set of lexemes belonging to a token.

A lexeme is the actual sequence of characters in the source program that matches a token's pattern.

What is meant by left recursion in a grammar?

A grammar is left-recursive if it contains at least one non-terminal A such that a derivation $A \rightarrow Ax$ is possible, where x is any string of grammar symbols. It explicitly means the non-terminal expands to a production where the very first symbol on the right-hand side is the same non-terminal.

	Syntax Analysis-----	Semantic
Analysis		

Purpose	Syntax analysis Checks if the code Sequence Follows the Grammar Rules-----
Semantic Analysis	Checks if the code sequence Is Logically valid or Meaningful

Input	Syntax analyzer takes the stream of tokens as it's input-----	Semantic
	analyzer takes the Parse Tree or Abstract Syntax Tree as input	

Output	Syntax Analyzer outputs the parse tree or abstract Syntax tree-----
Semantic Analyzer	outputs the validated Parse tree and populated symbol table

Error Check Checks errors such as missing semicolons or unmatched brackets-----
Checks errors such as type mismatching, or using and undeclared variable.

Distinguish between top-down parsing (LL) and bottom-up parsing (LR).

Top-Down Parsing (LL)	-----	Bottom-Up Parsing (LR)
Builds the parse tree from root to leaves.	-----	Builds the parse tree from leaves up to the root.

Scans left-to-right, constructs a Leftmost derivation.	-----	Scans left-to-right, constructs a Rightmost derivation in reverse.
--	-------	--

Cannot handle left-recursive grammars directly.	-----	Easily handles left recursion.
---	-------	--------------------------------

Less powerful, accepts a smaller class of grammars.	-----	More powerful, parses nearly all programming languages.
---	-------	---

Distinguish between NFA and DFA in the context of lexical analysis.

DFA (Deterministic Finite Automaton): In lexical analysis, DFAs are preferred for actual token matching implementation because they are fast; they execute precisely one state transition per

input character ($O(N)$ time). However, DFAs can be complex and memory-intensive to construct directly from Regular Expressions.

NFA (Nondeterministic Finite Automaton): NFAs are much easier to construct directly from regular expressions (via Thompson's construction) because they allow ϵ -transitions and multiple paths. However, simulating them is slower, so compiler generators (like Lex/Flex) typically generate NFAs first, then convert them to minimized DFAs for the final lexer.

DFA. In lexical analysis, DFAs are preferred for actual token matching implementation because they're fast. They execute exactly one state transition per input character ($O(n)$ time). However, DFAs can be complex and memory intensive to build directly from regular expression.

NFAs are much easier to build directly from regular expressions, because they allow epsilon transition and multiple paths, However, Simulating them is much slower, so compiler generator(e.g. lex,flex), Generates the NFA at first then Converts them into minimized DFAs for the final lexer.

Type conversion is the process of converting a value from one data type to another to ensure in an expression variables are evaluated consistently.

Token is an abstract symbol representing a lexical unit. They represent the meaningful sequence of characters in the source program. E.g. Keywords, identifiers and operators.

A pattern is a rule describing the set of lexemes belonging to a token.

Lexemes are the actual sequence of characters in the source program that matches a token's pattern.

Phases of Compiler:

1. Analysis Phase

2. Synthesis Phase

1.

a. Lexical Analysis: Converts the source code into streams of tokens.

b. Syntax Analysis: Checks if the code sequence matches the grammar rules and generates parse Tree or AST.

c. Semantic Analysis: Checks if the code sequence is logically valid and meaningful. Generates Annotated parse tree and symbol table.

d. Intermediate Code Generation: Generates the intermediate code from the annotated parse tree. E.G. Generating Three address Code, Quadraples, Triples etc.

2.

a. Code optimizer: Code optimization is the process of improving the intermediate code so that the target code runs faster, uses less memory, and more efficient.

Key techniques:

i. Eliminating dead Code.

ii. Loop optimization.

iii. Common subexpression elimination.

b. Target code generator: Generates the target code to execute, for example generating assembly code.

Symbol Table: is a data structure used in compiler design that stores the information about identifiers. used as a database for the compiler.

Error handler: handles lexical, syntax or semantic error.